Java Reference Notes

Unit 1: Fundamentals of OOP and Java Introduction

1. The Object-Oriented Paradigm

The **Object-Oriented Paradigm** is a software development model that organizes software design around data, or objects, rather than functions and logic.

- **Basic Principles (The Four Pillars):**
    1. **Encapsulation:** The process of binding data and the methods that manipulate them into a single unit (Class). It implements **Data Hiding** by making instance variables private and providing public getter/setter methods.
    2. **Inheritance:** A mechanism where a new class (Subclass/Derived) is created from an existing class (Superclass/Base). It supports the concept of **hierarchical classification** and eliminates redundant code.
    3. **Polymorphism:** Derived from Greek, meaning "many forms."
        - *Compile-time:* Method Overloading.
        - *Runtime:* Method Overriding (Dynamic Method Dispatch).
    4. **Abstraction:** The act of representing essential features without including background details. In Java, this is achieved using **Abstract Classes** and **Interfaces**.
- **Advantages of OOP:**
    - **Modularity:** Troubleshooting is easier because objects can be debugged independently.
    - **Reusability:** Through inheritance, we can use code from existing classes.
    - **Security:** Through encapsulation, vital data is hidden from unauthorized access.

    - **Flexibility:** Polymorphism allows a single interface to be used for a general class of actions.
- **Applications of OOP:** Real-time systems, Simulation and Modeling, Object-oriented databases, and AI-based expert systems.

2. Introduction to Java Language

- **Java History:** Developed by James Gosling at Sun Microsystems in 1995. Originally called "OAK." It was designed for handheld devices and set-top boxes but shifted focus to the World Wide Web.
- **Java Features (The "Buzzwords"):** * **Simple:** Removes complex features of C++ like pointers and operator overloading.
    - **Architectural Neutral:** The compiler generates **Bytecode** (.class files), which is not tied to any specific machine architecture.
    - **Portable:** You can carry the Java bytecode to any platform.
    - **Distributed:** Designed for the internet environment.
- **Difference between C, C++, and Java:**

Feature	**C**	**C++**	**Java**
Paradigm	Procedural	Hybrid (Pop + OOP)	Pure OOP
Pointers	Supported	Supported	Not supported (for security)
Memory Mgmt	Manual	Manual	Automatic (Garbage Collection)
Platform	Dependent	Dependent	Independent (Bytecode)

- **Java Environment:**
    - **JDK (Java Development Kit):** The full toolset (Compiler, Debugger, JRE).

    - **JRE (Java Runtime Environment):** Provides the libraries and JVM to run the code.
    - **JVM (Java Virtual Machine):** The engine that interprets Bytecode into machine code.
    - **API (Application Programming Interface):** A large collection of ready-made software components (packages).
- **Java Program Structure:**
    1. Documentation Section (Comments).
    2. Package Statement (Optional).
    3. Import Statements.
    4. Interface Statement.
    5. Class Definition.
    6. **Main Method Class:** Every Java application must have a public static void main(String args[]).

3. Java Programming Fundamentals

- **Java Tokens:** Smallest individual units in a program: Reserved Keywords, Identifiers, Literals, Operators, and Separators.
- **Data Types:** * **Primitive:** byte, short, int, long, float, double, char, boolean.
    - **Non-Primitive:** Classes, Interfaces, Arrays.
- **Type Casting:** * **Widening (Implicit):** Converting smaller type to larger (e.g., int to double).
    - **Narrowing (Explicit):** Converting larger to smaller (e.g., double to int) using (int).
- **Control Structures:**
    - **Decision Making:** if-else, switch-case.
    - **Looping:** while, do-while, for, and **Enhanced for-loop** (for-each).

- **Jumping:** break (exit loop), continue (skip iteration), return (exit method).

Unit 2: Classes, Objects, and Arrays

1. Basics of Classes and Objects

- **Class:** A logical entity. It contains **Fields** (variables) and **Methods**.
- **Object:** A physical entity. Created using the new keyword.
  - *Syntax:* ClassName objectName = new ClassName();

2. Constructors

A constructor initializes the object memory.

- **Rules:** Same name as class, no return type (not even void), called automatically during object creation.
- **Constructor Overloading:** Defining multiple constructors with different argument lists (type, number, or order of parameters).

3. Access Modifiers & Keywords

- **Static Members:** Variables or methods that belong to the **class** rather than the object. They can be called without creating an object.
- **'this' keyword:** Used to refer to the current class instance variable. It is also used to call one constructor from another (Constructor Chaining).

4. Arrays and Strings

- **One-Dimensional Arrays:** int arr[] = new int[5];
- **Two-Dimensional Arrays:** Used for matrices. int matrix[][] = new int[3][3];
- **Array of Objects:** Creating an array that stores references to objects. Student s[] = new Student[5];
- **String Handling:** Strings in Java are **Objects**.

- **Important Functions:** length(), charAt(), concat(), substring(), toUpperCase(), toLowerCase(), equals(), and indexOf().
- *Note:* Strings are **Immutable** (cannot be changed once created). For mutable strings, use StringBuffer or StringBuilder.

Unit 3: Multithreading and Exception Handling

1. Multithreading in Java

Multithreading is a specialized form of multitasking that allows a single program to perform multiple tasks concurrently. It is highly efficient because threads share the same memory area.

- **Concepts of Thread:** A thread is a "lightweight" process. It is the smallest unit of processing that can be performed in an OS.
- **Thread Life Cycle (States):**
  1. **New:** The thread is in this state after creation but before the start() method is called.
  2. **Runnable:** After start(), the thread is ready for execution and waiting for the Thread Scheduler.
  3. **Running:** The CPU is currently executing the thread's run() method.
  4. **Blocked/Waiting:** The thread is alive but not eligible to run (e.g., waiting for I/O or a lock).
  5. **Terminated (Dead):** The thread has finished execution.
- **Creating Threads:**
  1. **Extending Thread Class:** Create a class that extends Thread and override the run() method.
  2. **Implementing Runnable Interface:** Create a class that implements Runnable, pass its instance to a Thread object, and start it. (Preferred for multiple inheritance).
- **Thread Synchronization:** In multithreading, multiple threads may try to access the same resource simultaneously, leading to data inconsistency.

We use the synchronized keyword to ensure that only one thread can access a resource at a time.

- **Thread Priority:** Every thread has a priority (from 1 to 10). The Thread Scheduler uses this to decide which thread to run. Constants: MIN_PRIORITY (1), NORM_PRIORITY (5), MAX_PRIORITY (10).

2. Exception Handling

An **Exception** is an abnormal condition that arises in a code sequence at runtime. Java's exception handling is built around five keywords: try, catch, throw, throws, and finally.

- **Types of Exceptions:**
    1. **Checked Exceptions:** Checked at compile-time (e.g., IOException, SQLException).
    2. **Unchecked Exceptions:** Checked at runtime (e.g., ArithmeticException, NullPointerException).
- **Exception Handling Blocks:**
    - **Try:** Wraps the code that might throw an exception.
    - **Catch:** Handles the exception thrown by the try block. Multiple catch blocks can follow a single try (Specific exceptions first, then general Exception class).
    - **Finally:** A block that *always* executes regardless of whether an exception occurred. Used for cleanup (closing files/sockets).
    - **Throw:** Used to manually throw an exception.
    - **Throws:** Used in a method signature to declare that this method might throw certain exceptions, passing the responsibility to the caller.
- **Nested Try:** A try block inside another try block. Used when a part of a block may cause one error and the entire block itself may cause another.
- **User-Defined Exceptions:** You can create your own exception class by extending the Exception class.

Unit 4: File Handling and GUI Programming

1. File Handling (I/O)

Java performs I/O through **Streams**—a logical sequence of data.

- **I/O Streams Hierarchy:**
  - **Byte Streams:** Used for handling 8-bit bytes (binary data).
    - **Classes:** InputStream and OutputStream are the roots.
    - **Specifics:** FileInputStream (read from file) and FileOutputStream (write to file).
  - **Character Streams:** Used for handling 16-bit Unicode characters (text data).
    - **Classes:** Reader and Writer.
- **Object I/O (Serialization):** The process of writing the state of an object into a byte stream so it can be saved to a file or sent over a network. Uses ObjectInputStream and ObjectOutputStream.

2. Event Handling

Event handling is the mechanism that controls the event and decides what should happen if an event occurs.

- **Mechanism (Delegation Event Model):**
  1. **Source:** The UI component (Button, Checkbox) that generates the event.
  2. **Event Object:** Encapsulates the state changes in the source (e.g., ActionEvent).
  3. **Listener:** An interface that receives the event and processes it. The source must register the listener using addXListener() methods.
- **Event Types:** Foreground events (Button click, Menu selection) and Background events (OS interrupts, hardware failure).

- **Keyboard & Mouse Handling:** Handled via KeyListener (keyTyped, keyPressed) and MouseListener (mouseClicked, mouseEntered, etc.).

3. GUI Programming with AWT

AWT (Abstract Window Toolkit) is Java's original windowing library.

- **AWT Hierarchy:** * Component is the superclass of all UI elements.
    - Container is a component that can hold other components (e.g., Panel, Window).
    - Frame is a top-level window with a title and border.
- **Controls:**
    - **Labels:** To display text.
    - **Buttons:** To trigger actions.
    - **TextField/TextArea:** For user input.
    - **Checkboxes/Choice:** For selections.
- **Layout Managers:** Objects that control the size and position of components within a container.
    1. **FlowLayout:** Arranges components in a line (Default for Panels).
    2. **BorderLayout:** Arranges components in North, South, East, West, and Center (Default for Frames).
    3. **GridLayout:** Arranges components in a rectangular grid of equal-sized cells.